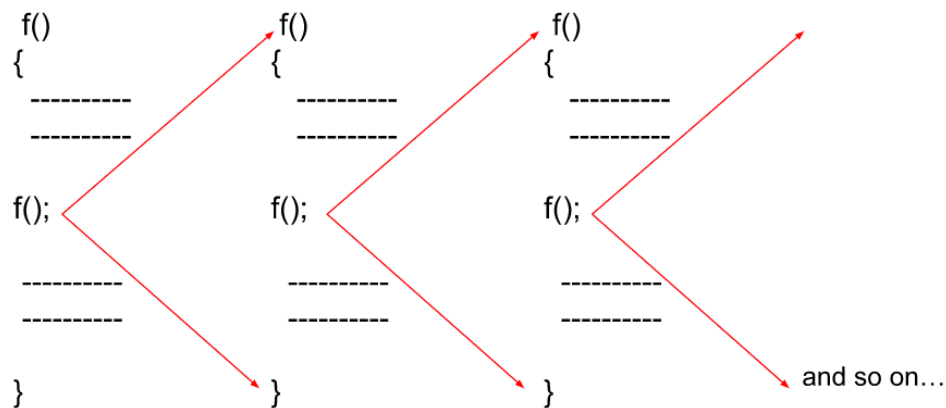


What is Recursion?

It is a phenomenon when a function calls itself indefinitely until a specified condition is fulfilled.

Let's understand recursion with the help of an illustration:



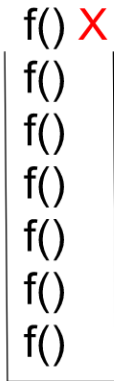
As we can see in the above image, a function is calling the same function inside its body. Since there is no condition to stop the recursive calls, the calls will run indefinitely until the stack runs out of memory (stack overflow).

What is Stack Overflow in Recursion?

Whenever recursion calls are executed, they're simultaneously stored in a **recursion stack** where they wait for the completion of the recursive function. A recursive function can only be completed if a base condition is fulfilled and the control returns to the parent function.

But, when there is no base condition given for a particular recursive function, it gets called indefinitely which results in a Stack Overflow i.e, exceeding the memory limit of the recursion stack and hence the program terminates giving a Segmentation Fault error.

The illustration above also represents the case of a Stack Overflow as there is no terminating condition for recursion to stop, hence it will also result in a memory limit exceeded error.



Base Condition: It is the condition that is written in a recursive function in order for it to get completed and not to run infinitely. After encountering the base condition, the function terminates and returns back to its parent function simultaneously.

To get a better understanding of how the base condition is an integral part of recursive functions, let us see an example below:

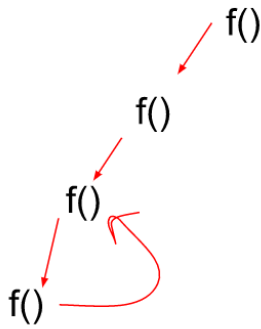
Let's say we have to print integers starting from 0 till 2 only, this will be how the pseudocode for it will look like

```
int count = 0;  
void func(){  
    if(count == 3 ) return;  
    print(count);  
    count++;  
    func();  
}
```

According to this pseudocode, the function will increment and print the value of count and then return when the base condition becomes true i.e, it will only print 0,1,2 and 3 and then execution gets completed.

Recursive Tree

A recursive tree is basically a representative form of recursion which depicts how functions are called and returned as a series of events happening consecutively. It is a pictorial description of the process of recursion as illustrated below:



When a recursive call gets completed, the control returns back to its parent function which is then further executed until the last function waiting in the recursive stack returns.

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward.

Alphanumeric characters include letters and numbers.

Given a string *s*, return true if it is a *palindrome*, or false otherwise.

Example 1:

Input: *s* = "A man, a plan, a canal: Panama"

Output: true

Explanation: "amanaplanacanalpanama" is a palindrome.

Approach & Thought Process (Palindrome Check with Two Pointers + Recursion)

Problem Understanding

We are given a string and need to determine whether it is a **palindrome**, considering **only alphanumeric characters** and **ignoring case differences**.

A palindrome reads the same **forward and backward** after filtering irrelevant characters.

Key Observations

1. Characters that are **not letters or digits** do not affect palindrome property.
 2. Uppercase and lowercase versions of the same character should be treated as equal.
 3. Comparison must proceed from **both ends of the string** toward the center.
-

Core Idea

Use the **two-pointer technique**, where:

- One pointer starts from the **left end**
- Another pointer starts from the **right end**

These pointers move inward while validating the palindrome condition.

Thought Process Behind the Solution

1 Base Condition

If the left pointer crosses or meets the right pointer:

- All necessary comparisons have already been made
- The string satisfies palindrome conditions

So, the string is considered a palindrome.

2 Handling Non-Alphanumeric Characters

If the character at the left pointer is **not alphanumeric**:

- It is ignored
- The left pointer is moved forward

Similarly, if the character at the right pointer is **not alphanumeric**:

- It is ignored
- The right pointer is moved backward

This ensures that only valid characters participate in comparisons.

3 Case-Insensitive Comparison

When both characters are alphanumeric:

- Convert both to the same case (lowercase)
- Compare them

If they differ:

- The string cannot be a palindrome

- The process terminates early
-

Recursive Progression

If the characters match:

- Move both pointers inward
- Repeat the same logic for the remaining substring

This recursive shrinking of the problem continues until a conclusion is reached.

Why Recursion Works Here

Each recursive call:

- Solves a **smaller subproblem**
- Moves closer to the center of the string
- Preserves the same logic at every step

Thus, the solution naturally fits a **divide-and-conquer** style.

Time and Space Complexity (Theoretical)

Time Complexity

- $O(n)$
Each character is visited at most once.

Space Complexity

- $O(n)$ (due to recursion stack in the worst case)

Get Strong Hold

Implement the `myAtoi(string s)` function, which converts a string to a 32-bit signed integer.

The algorithm for `myAtoi(string s)` is as follows:

1. **Whitespace:** Ignore any leading whitespace (" ").

2. **Signedness:** Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. **Conversion:** Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. **Rounding:** If the integer is out of the 32-bit signed integer range $[-2^{31}, 2^{31} - 1]$, then round the integer to remain in the range. Specifically, integers less than -2^{31} should be rounded to -2^{31} , and integers greater than $2^{31} - 1$ should be rounded to $2^{31} - 1$.

Return the integer as the final result.

Input: s = " -042"

Output: -42

Explanation:

Step 1: "___-042" (leading whitespace is read and ignored)

Step 2: " _-042" ('-' is read, so the result should be negative)

Step 3: " _-042" ("042" is read in, leading zeros ignored in the result)

◆ Problem Goal (Mental Model)

Convert a string into a **32-bit signed integer**, behaving like a real parser:

- Ignore leading spaces
- Read sign
- Read digits
- Stop at invalid character
- Clamp result if it overflows

The challenge is **over-flow handling**.

🟢 Optimal Approach

💡 Core Idea

"Never allow overflow to happen in the first place."

How you think optimally

1. Skip leading spaces
2. Read the sign and **store it separately**
3. Build the number **digit by digit**
4. **Before adding each digit**, check:
 - Will multiplying by 10 overflow?
 - Will adding this digit cross the boundary?
5. If overflow is inevitable:
 - Clamp immediately based on sign
6. Stop reading when a non-digit appears
7. Apply sign and return

Key Insight

Overflow occurs at this step:

$\text{result} = \text{result} * 10 + \text{digit}$

So, the correct question is:

“Is it safe to do this operation?”

That’s why:

- We divide by 10 to find the **last safe value**
- We check the last digit separately

Time & Space

- Time: $O(n)$
- Space: $O(1)$

Implement $\text{pow}(x, n)$, which calculates x raised to the power n (i.e., x^n).

Example 1:

Input: $x = 2.00000$, $n = 10$
Output: 1024.00000

Example 2:

Input: $x = 2.10000$, $n = 3$
Output: 9.26100

Example 3:

Input: $x = 2.00000$, $n = -2$
Output: 0.25000
Explanation: $2^{-2} = 1/2^2 = 1/4 = 0.25$

Brute-Force Approach (Thought Process)

Initial Human Intuition

“Power means multiply the number by itself n times.”

This is the **most direct interpretation** of exponentiation.

How the brute-force solution is designed

1. If n is negative:
 - Convert it to positive using absolute value
 - Plan to take the reciprocal at the end
 2. Start with a result initialized to 1
 3. Multiply x repeatedly:
 - Do this exactly $|n|$ times
 4. If the original exponent was negative:
 - Return the reciprocal of the result
-

Why this works

- It follows the **definition** of exponentiation
 - Simple to understand and implement
 - Correct for all valid inputs
-

✗ Why this is inefficient

- Requires n multiplications
 - If n is large (e.g., 10^9), it becomes impractical
 - Wastes computation by repeating the same multiplications
-

🧠 Complexity (Brute Force)

- **Time:** $O(n)$
 - **Space:** $O(1)$
-

💡 Optimal Approach (Thought Process)

💡 Key Observation

Exponentiation has **hidden mathematical structure** that can reduce work.

🧠 Core Insight (Binary Exponentiation)

$$x^n = \begin{cases} (x^2)^{n/2}, & \text{if } n \text{ is even} \\ x \cdot x^{n-1}, & \text{if } n \text{ is odd} \end{cases}$$

This allows us to:

- **Square the base**
- **Halve the exponent**

Each step dramatically reduces the problem size.

🧠 How the optimal solution is designed

1. Convert exponent n to a positive value
2. Maintain a running result initialized to 1
3. While exponent is greater than zero:
 - If exponent is **even**:

- Square the base
 - Divide the exponent by 2
 - If exponent is **odd**:
 - Multiply result by base
 - Reduce exponent by 1
 - 4. If original exponent was negative:
 - Return reciprocal of result
-

Why this is optimal

- Every step reduces the exponent significantly
 - Number of operations becomes logarithmic
 - Avoids redundant multiplications
-

Complexity (Optimal)

- **Time:** $O(\log n)$
- **Space:** $O(1)$

A digit string is **good** if the digits (**0-indexed**) at **even** indices are **even** and the digits at **odd** indices are **prime** (2, 3, 5, or 7).

- For example, "2582" is good because the digits (2 and 8) at even positions are even and the digits (5 and 2) at odd positions are prime. However, "3245" is **not** good because 3 is at an even index but is not even.

Given an integer n , return *the total number of good digit strings of length n* . Since the answer may be large, **return it modulo $10^9 + 7$** .

A **digit string** is a string consisting of digits 0 through 9 that may contain leading zeros.

Input: $n = 1$

Output: 5

Explanation: The good numbers of length 1 are "0", "2", "4", "6", "8".

1. Brute Force Thought Process (Exploring Every String)

Imagine we build the number digit by digit.

For each index:

- If index is **even** → we can place one of 5 digits {0,2,4,6,8}
- If index is **odd** → we can place one of 4 digits {2,3,5,7}

So at every position we make a choice.

If we think recursively:

- At index 0 → 5 branches
- At index 1 → each branch splits into 4
- At index 2 → each splits into 5
- At index 3 → each splits into 4
- ... and so on

So the recursion tree grows like:

$5 \times 4 \times 5 \times 4 \times \dots$

This means we are **constructing every possible valid string**.

Why This Is Inefficient

If length = n:

Even positions count = $\lceil n/2 \rceil$

Odd positions count = $\lfloor n/2 \rfloor$

Total strings explored:

$$5^{\lceil n/2 \rceil} \times 4^{\lfloor n/2 \rfloor}$$

This is **exponential growth**.

So brute force complexity:

$$O(5^{n/2} \cdot 4^{n/2})$$

This becomes impossibly large very quickly.

We are doing unnecessary work because:

👉 We don't need the strings.

👉 We only need the count.

🧠 2. Key Observation (Shift in Thinking)

Instead of asking:

"How do I generate all valid numbers?"

We ask:

"How many choices do I have at each position?"

This is a fundamental counting principle:

If a task consists of independent steps,
and each step has some number of choices,
then total ways = multiplication of choices.

🧠 3. Applying Multiplication Principle

Each position is independent.

- Every even position has 5 choices.
- Every odd position has 4 choices.

So total good numbers:

$$5^{\text{even positions}} \times 4^{\text{odd positions}}$$

Now we just need to count how many even and odd positions exist.

🧠 4. Why $\lceil n/2 \rceil$ and $\lfloor n/2 \rfloor$?

Indexes go from 0 to $n-1$.

Since we start at index 0 (even index):

- If n is even $\rightarrow$ even and odd positions are equal.
- If n is odd $\rightarrow$ there is one extra even index.

So:

Even positions = $\lceil n/2 \rceil$

Odd positions = $\lfloor n/2 \rfloor$

This automatically handles both even and odd n .

5. Now the Real Challenge

We need to compute:

$$5^{\lceil n/2 \rceil} \times 4^{\lfloor n/2 \rfloor}$$

But n can be very large.

Direct multiplication would take too long.

So, we use a mathematical trick:

6. Idea of Fast Exponentiation

Instead of multiplying the base exponent times:

$$a^n = a \times a \times a \times \dots (n \text{ times})$$

We use:

$$a^n = (a^{n/2})^2$$

Each time exponent is cut in half.

So, the number of operations becomes proportional to:

$$\log n$$

This is the key optimization.

Final Complexity

Brute Force:

Time $\rightarrow$ Exponential

Space $\rightarrow$ Linear (recursion depth)

Optimized Approach:

Time $\rightarrow O(\log n)$

Space $\rightarrow O(1)$

You are given a stack of integers. Your task is to sort the stack in descending order using recursion, such that the top of the stack contains the greatest element. You are not allowed to use any loop-based sorting methods (e.g., quicksort, mergesort). You may only use recursive operations and the standard stack operations (push, pop, peek/top, and isEmpty).

Optimal approach

There are two main ideas:

1 Break the stack down completely

Keep removing the top element until the stack becomes empty.

Why?

Because an empty stack is already sorted.

That becomes the base case.

So the thinking is:

“If I can sort a smaller stack, then I can sort the full stack.”

2 Insert elements back in sorted order

After reaching the empty stack, we begin rebuilding it.

Each removed element is inserted back into its correct position in the already-sorted smaller stack.

The idea here is:

“To insert an element into a sorted stack, temporarily remove larger elements until the correct position is found, insert the element, and then put the removed elements back.”

This ensures the stack remains sorted at every step while rebuilding.

Conceptual Flow

1. Remove the top element.
2. Recursively sort the remaining stack.

3. Insert the removed element into its correct sorted position.

This is conceptually very similar to **insertion sort**, but implemented recursively using stack operations.

Why This Approach Works

- Recursion allows us to reduce the problem to the smallest possible case (empty stack).
- While returning from recursion, elements are inserted in sorted order.
- The recursive call stack temporarily stores elements just like an auxiliary stack would.

So effectively:

- We remove everything.
 - Then rebuild the stack in sorted order.
-

Time Complexity Analysis

Let **n** be the number of elements in the stack.

- The sorting function is called once per element → about **n recursive calls**.
- For each element, inserting it in the correct position may require traversing most of the stack → up to **n operations** in the worst case.

So overall:

$$T(n) = T(n-1) + O(n)$$

Which results in:

✓ **Time Complexity: $O(n^2)$**

Worst case occurs when the stack is sorted in reverse order.

Space Complexity Analysis

No extra data structures are used, but recursion consumes call stack space.

- Maximum recursion depth = **n**

✓ **Auxiliary Space Complexity: $O(n)$**

You are given a stack of integers. Your task is to reverse the stack using recursion. You may only use standard stack operations (push, pop, top/peek, isEmpty). You are not allowed to use any loop constructs or additional data structures like arrays or queues.

Optimal

♦ Core Idea

The solution works in two conceptual phases:

1 Remove Everything (Break Phase)

Keep removing the top element until the stack becomes empty.

Why?

Because once the stack is empty, there is nothing left to reverse — that's the base case.

So mentally:

"If I can reverse a smaller stack, I can reverse the whole stack."

2 Insert Each Element at the Bottom (Rebuild Phase)

After recursion empties the stack, we start rebuilding it.

But instead of pushing elements back on top (which would preserve order), we insert each removed element **at the bottom**.

This causes the order to flip.

🔄 How the Reversal Actually Happens (Conceptually)

Suppose the stack (top on right) is:

[1, 2, 3, 4]

Step 1: Remove everything recursively

- Remove 4
- Remove 3
- Remove 2

- Remove 1
- Stack becomes empty

Step 2: Rebuild by inserting at bottom

- Insert 1 $\rightarrow$ [1]
- Insert 2 at bottom $\rightarrow$ [2, 1]
- Insert 3 at bottom $\rightarrow$ [3, 2, 1]
- Insert 4 at bottom $\rightarrow$ [4, 3, 2, 1]

The stack is now reversed.

Key Insight

Normal stack behavior is **LIFO (Last In, First Out)**.

To reverse it:

- Remove elements in LIFO order.
- Reinsert them in such a way that the earliest removed element ends up on top.

By always inserting at the bottom during the rebuild phase, we invert the order.

Time Complexity Analysis

Let **n** be the number of elements in the stack.

Reverse operation:

- reverseStack is called once per element $\rightarrow$ **n calls**

Insert at bottom:

- Each insertion may require removing up to **n elements** in the worst case.

So total work:

$$T(n) = T(n-1) + O(n)$$

Which expands to:

 **Time Complexity: $O(n^2)$**

Worst case happens because for each of the n elements, we may traverse the entire stack.

Space Complexity Analysis

- No extra data structures are used.
- But recursion depth goes up to n .

✓ **Auxiliary Space Complexity: $O(n)$**

(due to recursive call stack)

Subsequence Patterns

Given an integer n . You need to generate all the binary strings of n characters representing bits.

Note: Return the strings in ascending order.

Input: $n = 2$

Output: [00, 01, 10, 11]

Explanation: As each position can be either 0 or 1, the total possible combinations are 4.

◆ Core Idea (Decision-Based Thinking)

At every position in the string:

- Decide whether to place '0' or '1'
- Move to the next position
- Repeat until the string reaches length n

This creates a **binary decision tree**, where:

- Each level represents one character position
 - Each node branches into two choices (0 and 1)
-

◆ Why Recursion Fits Naturally

Recursion is ideal because:

- The problem is **self-similar**:
generating a string of length k depends on generating a string of length $k-1$
- Each recursive call handles **one position**
- The recursion depth is exactly n

So recursion maps directly to the structure of the problem.

◆ Role of StringBuilder

- We build the binary string **character by character**
- StringBuilder is used because it is **mutable** and efficient
- The same builder is reused across recursive calls to avoid creating new objects repeatedly

However, since it is mutable, **careful undoing (backtracking)** is required.

◆ Backtracking Thought Process

1. Add a character ('0' or '1')
2. Explore all possibilities from that choice
3. Remove the character before trying the next choice

This ensures:

- Each recursive call starts with a **clean state**
- Changes made in one branch do not affect others

Every decision is temporary unless it leads to a complete valid solution.

◆ Base Case Intuition

When the number of chosen characters equals n :

- We have formed one complete binary string
- This string is added to the result list
- The recursion stops for that path

This guarantees:

- Only valid strings of exact length n are stored
- No partial or extra-length strings are included

◆ How All Strings Are Covered

Because:

- Every position tries both '0' and '1'
- Every path reaches exactly length n

The algorithm explores **all 2^n combinations** without repetition.

◆ Complexity (Conceptual)

- **Time Complexity:** $O(2^n)$
 - Every possible binary string is generated
- **Space Complexity:** $O(n)$
 - Maximum recursion depth
 - Output storage requires $O(2^n)$

Given an integer array `nums` of unique elements, return *all possible subsets (the power set)*.

The solution set must not contain duplicate subsets. Return the solution in any order.

Problem Understanding (Common for both)

A **subsequence** means:

- You can either **pick** an element or **not pick** it
- Order of elements must remain the same
- For an array of size n , total subsequences = 2^n

So for {1, 2, 3}, every element has **2 choices** → include or exclude.

1 Recursive Approach

Core Idea

At every index, we make a **binary decision**:

- **Take** the current element
- **Do not take** the current element

This naturally forms a **decision tree**.

Thought Process

1. Start from index 0
2. For each element:
 - Choice 1 → Include the element in the current subsequence
 - Choice 2 → Exclude the element
3. Move to the next index after making a choice
4. When we reach the end of the array:
 - Whatever choices we made so far form **one valid subsequence**
 - Store it

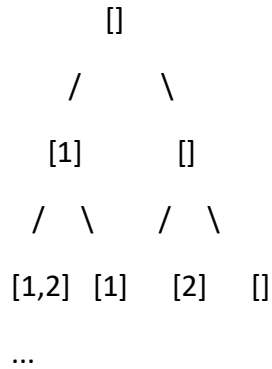
This guarantees that **all combinations of include/exclude** are explored.

Why Backtracking is Needed

- We use **one list** to build subsequences
- After exploring the “include” path, we must **undo the choice**
- This allows us to correctly explore the “exclude” path
- This undo step is called **backtracking**

Recursion Tree Insight

For {1,2,3}:



Each level represents a decision for one element.

Time & Space Complexity

- **Time Complexity:** $O(2^n)$
 - Every possible subsequence is generated
 - **Space Complexity:** $O(n)$ (recursion stack)
 - Plus storage for results: $O(2^n * n)$ overall
-

2 Iterative Approach

Core Idea

Build subsequences **level by level**, similar to **Breadth First expansion**.

Thought Process

1. Start with an **empty subsequence**
 - This represents “no elements chosen yet”
2. For each element in the array:
 - Take all existing subsequences
 - Create **new subsequences by adding the current element**
 - Append these new ones to the list

3. Repeat for all elements

Each step **doubles** the number of subsequences.

Why This Works

- Initially: 1 subsequence $\rightarrow []$
- After first element: 2 subsequences
- After second element: 4 subsequences
- After third element: 8 subsequences

This matches the formula 2^n

Conceptual Visualization

For {1,2,3}:

Start: [[]]

Add 1 $\rightarrow [[], [1]]$

Add 2 $\rightarrow [[], [1], [2], [1,2]]$

Add 3 $\rightarrow [[], [1], [2], [1,2], [3], [1,3], [2,3], [1,2,3]]$

Each new element is **added to all existing subsets**.

Time & Space Complexity

- **Time Complexity:** $O(2^n)$
- **Space Complexity:** $O(2^n * n)$

Given n pairs of parentheses, write a function to *generate all combinations of well-formed parentheses*.

```
Input: n = 3
```

```
Output: ["((()))", "(()())", "(())()", "()(())", "()()()"]
```

 **Core problem understanding**

For n pairs of parentheses:

- Total length of each string = $2n$
- Every valid string must satisfy **two rules**:
 1. At no point should $)$ be more than $($ (prefix condition)
 2. Total number of $($ must equal total number of $)$

So the entire problem is about **generating combinations** while respecting these constraints.

Approach 1: Brute Force + Validation

Intuition

“Let me generate **all possible strings** of length $2n$ using $($ and $)$ and then **filter out only the valid ones**.”

This is the most straightforward idea that comes to mind.

Thought process

1. At each position in the string, you have **two choices**:
 - Put $($
 - Put $)$
2. Since the length must be $2n$, the total combinations are:
3. $2^{(2n)}$

(most of which are invalid)

4. Once a string is fully built:
 - Check whether it is **valid**
 - If valid $\rightarrow$ keep it
 - If not $\rightarrow$ discard it
-

Why isValid() works conceptually

The validation logic uses a **balance counter**:

- $($ increases balance

- ')' decreases balance

Two conditions define validity:

1. Balance should **never go negative**
→ ensures no closing bracket appears before an opening one
2. Balance must be **zero at the end**
→ ensures all openings are properly closed

This mimics how a human checks parentheses while reading left to right.

✗ Why this approach is inefficient

- You **generate many useless strings**
- Validation happens **after full construction**
- Time complexity grows exponentially

👉 This solution is correct but **wastes a lot of work**

◆ Time Complexity

- At each of the $2n$ positions, there are **2 choices**: '(' or ')'
- Total strings generated: 2^{2n}
- For **each string**, validation (isValid) takes: $O(2n)$

✓ **Total Time Complexity:** $O(2^{2n} \cdot 2n) \approx O(n \cdot 4^n)$

◆ Space Complexity

1. **Recursion depth** of helper(): $O(2n)$
2. **StringBuilder space:** $O(2n)$
3. **Result storage:**
 - Number of valid combinations = **Catalan number**

$$C_n = \frac{2n!}{n!(n+1)!}$$

- Each string length = $2n$

✅ **Total Space Complexity:** $O(C_n \cdot 2n)$

(If output space is excluded, auxiliary space is $O(n)$.)

🚀 Approach 2: Backtracking (Optimal)

💡 Key intuition

“Why generate invalid strings at all?

Let’s **only build strings that are valid at every step.**”

This is the optimization mindset.

🧠 Thought process

Instead of validating at the end, we **enforce rules while building.**

We track:

- open $\rightarrow$ number of '(' used
 - close $\rightarrow$ number of ')' used
-

🔒 Constraints applied during construction

1. You can add '(' **only if:**
2. open $< n$

(you can’t exceed n opening brackets)

3. You can add ')' **only if:**
4. close $< open$

(you can’t close more than you’ve opened)

These two rules **guarantee correctness by design.**

🧩 Why this guarantees validity

- The prefix condition is **never violated**
- Total '(' and ')' are naturally balanced

- When length reaches $2n$, the string is **already valid**

So no extra validation is needed.

Conceptual advantage

This is **pruning the recursion tree**:

- Invalid paths are **never explored**
- Only meaningful branches are expanded

This is the essence of **backtracking**:

“Explore only what can lead to a valid solution.”

◆ Time Complexity

- Only **valid** parenthesis sequences are generated
- Number of valid sequences = **Catalan number**

$$C_n = \frac{2n!}{n!(n+1)!}$$

- Constructing each valid string takes $O(2n)$

✓ **Total Time Complexity:** $O(C_n \cdot 2n)$

◆ Space Complexity

1. **Recursion stack depth:** $O(2n)$
2. **Current string storage:** $O(2n)$

✓ **Total Space Complexity:** $O(C_n \cdot 2n)$

(Excluding output space $\rightarrow O(n)$)

Given an array `arr` of non-negative integers and an integer `target`, the task is to count all subsets of the array whose sum is equal to the given target.

Input: arr[] = [5, 2, 3, 10, 6, 8], target = 10

Output: 3

Explanation: The subsets {5, 2, 3}, {2, 8}, and {10} sum up to the target 10.

Problem Understanding

We are asked to **count the number of subsets** whose elements add up to a given target.

A **subset** means:

- We can pick or not pick each element
 - Order does **not** matter
 - Every element has an independent decision
-

Core Intuition

For every element in the array, we have **two independent choices**:

1. **Include** the element in the current subset
2. **Exclude** the element from the current subset

By exploring **both choices for every element**, we guarantee that **all possible subsets** are considered.

This is the most direct (brute-force) way to solve the problem.

Thought Process (Recursive Decision Making)

1. Start from the **first element** of the array.
2. Move forward **one index at a time**.
3. At each index:
 - Imagine one path where the element is included (sum increases)
 - Imagine another path where the element is excluded (sum stays the same)
4. Continue making these decisions until all elements are processed.

This forms a **decision tree** where:

- Each level corresponds to an element

- Each node branches into two possibilities
-

Base Condition Insight

Once all elements have been considered:

- We have formed **one complete subset**
- At this point, we simply check:
 - Does the accumulated sum equal the target?
- If yes, this subset is counted

This ensures **only valid subsets** contribute to the answer.

Conceptual Model (Recursion Tree)

- Depth of the tree = number of elements (n)
- Each node splits into **two branches**
- Total possible paths from root to leaves = **all subsets**

Thus, the algorithm systematically explores the **entire subset space**.

Why this approach is correct

- Every subset corresponds to **exactly one path** in the recursion tree
- No subset is skipped
- No subset is counted more than once

Hence, the count obtained is accurate.

Limitations (Theoretical Awareness)

- This method explores **all subsets**, even those that will never reach the target
- The number of subsets grows exponentially (2^n)
- Therefore, the approach is **correct but not efficient** for large inputs

This is why it's often the **starting point** before optimization using DP.

Time Complexity (TC)

- For **each element**, the algorithm makes **two recursive calls**:
 1. Include the element
 2. Exclude the element

This creates a **binary recursion tree**.

Number of recursive calls

- Depth of recursion = n (number of elements)
- Total nodes in recursion tree: 2^n

✓ Final Time Complexity: $O(2^n)$

Space Complexity (SC)

1 Recursion stack space

- Maximum depth of recursion = n
- Each stack frame holds a few integers: $O(n)$

Given an array of distinct integers candidates and a target integer target, return *a list of all unique combinations of candidates where the chosen numbers sum to target*. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Input: candidates = [2,3,6,7], target = 7

Output: [[2,2,3],[7]]

Explanation:

2 and 3 are candidates, and $2 + 2 + 3 = 7$. Note that 2 can be used multiple times.

7 is a candidate, and $7 = 7$.

These are the only two combinations.

Intuition

The goal of **Combination Sum** is to find **all unique combinations** of numbers that add up to a given target, where:

- Each number can be chosen **multiple times**
 - The **order of elements inside a combination does not matter**
 - Different combinations must be **unique**
-

◆ Key Observation 1: Decision at each index

For every element in the array, we face **two fundamental choices**:

1. **Take the current element**
2. **Skip the current element**

This naturally leads to a **decision tree**, which is why **recursion + backtracking** fits perfectly.

◆ Key Observation 2: Why index is not incremented when we “take”

When we **include arr[index]**, we are allowed to use it again.

So instead of moving forward, we **stay on the same index** and reduce the target.

This ensures:

- Unlimited usage of the same element
- No duplicate permutations (e.g. [2,3] and [3,2])

➡ This is the **core trick** that enforces uniqueness.

◆ Key Observation 3: Pruning invalid paths early

There are three stopping situations:

1. **Target becomes 0**
 - A valid combination is formed
 - Current path is stored
2. **Index reaches end of array**

- No more elements to choose
- Path cannot succeed

3. **Current element > target**

- Cannot include it
- Only option is to skip

This **pruning** avoids unnecessary recursive calls and keeps the search efficient.

◆ **Why backtracking works here**

- We **build the solution incrementally**
- Once a path is explored, we **undo the last choice**
- Then we explore alternative possibilities

This ensures:

- All valid combinations are explored
 - No extra memory is used for copying paths during recursion
-

🌳 **Conceptual Recursion Tree**

At each level:

```

      index
    /   \
  take  not take
  (same i) (i+1)
  
```

- Depth depends on how many times elements can be reused
 - Width depends on number of choices
-

🕒 **Time Complexity (TC)**

There is **no tight polynomial bound**, because:

- We must **generate all valid combinations**
- The number of combinations varies with input

Worst-case:

- Exponential in nature
- Approximately:

$O(2^T)$ (T = target, in worst case when smallest element is 1)

More formally: $O(\text{number of valid combinations} \times \text{average length of each combination})$

Space Complexity (SC)

1. Recursion stack

- Maximum depth occurs when the smallest element is repeatedly chosen
- $\text{Depth} \approx \text{target} / \min(\text{arr})$

$O(T / \min(\text{arr}))$

2. Temporary list (current)

- Stores elements of one combination
- Same as recursion depth

3. Result storage

- Space required to store all valid combinations
- Output-dependent (cannot be avoided)

Overall SC = $O(T / \min(\text{arr})) + O(\text{output size})$

Given a collection of candidate numbers (candidates) and a target number (target), find all unique combinations in candidates where the candidate numbers sum to target.

Each number in candidates may only be used once in the combination.

Note: The solution set must not contain duplicate combinations.

```
Input: candidates = [10,1,2,7,6,1,5], target = 8
```

```
Output:
```

```
[  
  [1,1,6],  
  [1,2,5],  
  [1,7],  
  [2,6]  
]
```

Intuition Behind the Solution

The problem asks us to find **all unique combinations** of numbers from the array such that their sum equals the given target, with the constraint that **each element can be used at most once**.

This naturally fits a **backtracking (DFS)** approach because:

- We need to **explore all possible subsets**.
- At every step, we make a **choice**: include a number or move on.
- Once the current path becomes invalid, we **backtrack**.

Core Thought Process (Step-by-Step)

1. Why Sorting Is Necessary

The array is sorted before starting recursion. This helps in two crucial ways:

- **Early stopping (pruning):**
If $\text{arr}[i] > \text{target}$, then all elements after it will also be greater $\rightarrow$ no need to continue.
- **Handling duplicates:**
Sorting places duplicates together, making it easy to skip repeated combinations.

2. Recursive State Definition

At any point, the recursion is defined by:

- **index**: where to start choosing elements from (ensures elements are used only once).
- **target**: remaining sum we need to achieve.
- **current**: the current combination being built.

This clearly represents a **decision tree** of choices.

3. Base Cases (Stopping Conditions)

✓ When `target == 0`

- We have found a valid combination.
- We **copy** the current list and store it in `res`.
- Copying is important because `current` will change during backtracking.

✗ When:

- `index == arr.length` (no elements left), or
- `arr[index] > target` (sum exceeded)

We stop exploring this path because it can no longer lead to a valid solution.

4. Exploring Choices (For Loop)

From the current index, we try **each possible candidate**:

- Add `arr[i]` to the current combination.
- Recurse with:
 - `i + 1` → ensures the same element is **not reused**
 - `target - arr[i]` → reduces remaining sum

After recursion:

- Remove the last element → **backtracking**
 - This allows exploring the next possibility cleanly.
-

5. Avoiding Duplicate Combinations

The condition:

```
if (i <= index || arr[i] != arr[i-1])
```

ensures that:

- At the **same recursion level**, duplicate numbers are not chosen again.
- This prevents generating the same combination in different orders.

Conceptually:

- **Duplicates are allowed across different levels**
- **Duplicates are skipped at the same level**

This guarantees **uniqueness of combinations**.

Why Backtracking Works Here

- The solution space is **combinatorial**
 - Each recursive call explores one decision
 - Backtracking ensures:
 - All valid paths are explored
 - Invalid paths are abandoned early (pruning)
-

Time Complexity

Let:

- n = number of elements
- k = average size of a valid combination

Recursive Exploration

- In the worst case, we explore **all subsets** $\rightarrow O(2^n)$

Copying the Result

- Each valid combination requires copying k elements
 $\rightarrow O(k)$ per valid solution

Overall Time Complexity: $O(2^n \cdot k)$

(This is typical for subset / combination generation problems.)

Space Complexity

- **Recursion stack:** $O(n)$ (depth of recursion)
- **Temporary list (current):** up to $O(k)$
- **Result storage:** depends on number of valid combinations

$$O(n + \text{output size})$$

Given a array arr of integers, return the sums of all subsets in the list. Return the sums in sorted order.

Input: arr[] = [2, 3]

Output: [0, 2, 3, 5]

Explanation: When no elements are taken then Sum = 0. When only 2 is taken then Sum = 2. When only 3 is taken then Sum = 3. When elements 2 and 3 are taken then Sum = 2+3 = 5.

Problem Intuition

We are asked to compute the **sum of elements for every possible subset** of a given array.

Key observation:

- Every element has **two choices**:
 - include it in the subset
 - exclude it from the subset
- Each unique sequence of choices forms a **unique subset**
- Since there are n elements, the total number of subsets is:

$$2^n$$

Core Thought Process

Instead of explicitly constructing subsets, we track **only the running sum**.

At every index:

- If we **exclude** the current element → sum remains unchanged
- If we **include** the current element → sum increases by that value

This leads to a **binary decision tree**, where:

- Each level corresponds to one array element
- Each root-to-leaf path represents one subset
- The leaf stores the final subset sum

Why Recursion Works Here

- The recursion explores **all inclusion/exclusion combinations**
- When we reach the end of the array:
 - all decisions are fixed
 - the accumulated sum corresponds to one subset
- This sum is stored in the result list

So the idea is:

“Explore all subsets implicitly, but store only their sums.”

Time Complexity

Recursive Calls

- Each element causes **two recursive calls**
- Total number of calls: 2^n

Work per Call

- Constant work (sum update and function call)

Sorting the Result

- Number of sums = 2^n
- Sorting cost: $O(2^n \log 2^n) = O(n \cdot 2^n)$

Final Time Complexity: $O(n \cdot 2^n)$

(Sorting dominates the recursion cost.)

Space Complexity

Recursion Stack

- Depth = n: $O(n)$

Result Storage

- Stores 2^n sums: $O(2^n)$
-

Final Space Complexity: $O(2^n)$

Given an integer array `nums` that may contain duplicates, return *all possible subsets (the power set)*.

The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

Input: `nums = [1,2,2]`

Output: `[[], [1], [1,2], [1,2,2], [2], [2,2]]`

Problem Intuition (Common to All)

We are asked to generate **all subsets** of an array that **may contain duplicates**, but the **result must contain only unique subsets**.

Key observations:

- Every element has **two choices**: include or exclude $\rightarrow 2^n$ total subsets.
 - Duplicates in the input create **identical recursion paths**, which must be handled carefully.
-

1 Brute Force Approach

Thought Process

- Treat the problem as a **binary decision tree**.
- For each element:
 - either include it
 - or exclude it
- This guarantees that **all possible subsets** are generated.

However:

- If the input has duplicate values, the same subset can be generated **multiple times**.
- To avoid storing duplicates, each generated subset is:
 - compared against all previously stored subsets
 - added only if it does not already exist.

So the idea is: "Generate everything first, then filter duplicates."

Why This Works

- Subset generation is exhaustive → correctness guaranteed.
 - Duplicate detection is done using `List.equals()`.
-

Time Complexity Analysis

- Number of generated subsets → 2^n
- At each leaf:
 - copying subset → $O(k)$ ($\leq n$)
 - checking `res.contains()`:
 - scans up to 2^n subsets
 - each comparison costs $O(k)$

Total time: $O(n \cdot 4^n)$

Space Complexity

- Recursion stack → $O(n)$
- Result storage → up to 2^n subsets of size n

$$O(n \cdot 2^n)$$

2 Better Approach Using HashSet

Thought Process

- The subset generation logic remains the same.
- Instead of manually checking for duplicates:
 - store subsets in a HashSet
- Hashing ensures:
 - fast lookup
 - automatic duplicate elimination

So the idea becomes: “Generate everything, but let hashing remove duplicates efficiently.”

Why This Is Better

- Avoids expensive linear scanning (contains)
 - Duplicate checking becomes near constant-time
-

Time Complexity Analysis

- Total subsets generated $\rightarrow 2^n$
- For each subset:
 - hashing + equality check $\rightarrow O(k) (\leq n)$

Total time: $O(n \cdot 2^n)$

Space Complexity

- Recursion stack $\rightarrow O(n)$
- HashSet storage $\rightarrow 2^n$ subsets

$$O(n \cdot 2^n)$$

Core Insight

Hashing reduces duplicate checking cost but does not prevent duplicate generation.

3 Optimal Approach

Thought Process

- Observe that duplicates only occur when **the same value is chosen multiple times at the same recursion level**.
- Sort the array so that duplicates are adjacent.
- While iterating:
 - allow the **first occurrence**
 - skip subsequent duplicates **at the same depth**

So the strategy is: "Prevent duplicates during generation itself."

Why This Is Optimal

- Duplicate subsets are **never generated**
 - No post-processing or filtering required
 - Each valid subset is created exactly once
-

Time Complexity Analysis

- Total unique subsets $\rightarrow 2^n$
- Copying each subset $\rightarrow O(k) (\leq n)$

Total time: $O(n \cdot 2^n)$

(Sorting adds $O(n \log n)$, which is dominated by 2^n .)

Space Complexity

- Recursion stack $\rightarrow O(n)$
- Result storage $\rightarrow O(n \cdot 2^n)$

Find all valid combinations of k numbers that sum up to n such that the following conditions are true:

- **Only numbers 1 through 9 are used.**

- Each number is used at most once.

Return *a list of all possible valid combinations*. The list must not contain the same combination twice, and the combinations may be returned in any order.

```
Input: k = 3, n = 7
Output: [[1,2,4]]
Explanation:
1 + 2 + 4 = 7
There are no other valid combinations.
```

Problem Intuition

We need to find **all unique combinations** of:

- exactly k numbers
 - chosen from the range 1 to 9
 - such that their **sum is exactly n**
 - each number is used **at most once**
 - combinations must be in **increasing order** (no permutations)
-

Core Thought Process

1. Fixed Search Space

The numbers are strictly limited:

- Only integers from 1 to 9
- This creates a **small and bounded decision space**

This immediately suggests **backtracking**, because:

- We must explore combinations under multiple constraints
 - Partial solutions can be rejected early
-

2. State Representation

At any point in the recursion, the state is defined by:

- the **last chosen number** (to maintain increasing order)

- the **current combination size**
- the **current sum**
- the **current combination**

This state fully describes a partial solution.

3. Decision Making

From a given state:

- We can only choose numbers **greater than the last chosen one**
- This ensures:
 - no duplicates
 - no permutations
- Each choice adds exactly one number to the combination

This turns the problem into a **combinations (not permutations)** search.

4. Base Conditions (Pruning Logic)

Two critical pruning rules are applied:

✗ If current sum exceeds target

- No future additions can reduce the sum
- This path is immediately abandoned

✓ If current size reaches k

- If the sum equals the target → valid combination
- Otherwise → invalid, stop exploring further

These checks dramatically reduce unnecessary recursion.

5. Why the Loop Stops Early

Since numbers are chosen in increasing order:

- Once $\text{sum} + i > \text{target}$, all further i will also exceed the target
- So the loop terminates early

This is a classic **branch-and-bound** optimization.

Why Backtracking Is Optimal Here

- The constraints are **tight**
- Partial solutions can be evaluated early
- Only valid candidates are extended
- Each combination is generated **exactly once**

So the algorithm:

“Build combinations incrementally, abandon invalid paths early, and record only complete valid ones.”

Time Complexity

Let:

- The search space be numbers 1 to 9
- Maximum combination size be k

Upper Bound

- Total subsets of $\{1..9\} \rightarrow 2^9 = 512$
- Each subset is checked at most once

Thus:

$$O(2^9) \approx O(1)$$

From a practical standpoint, this is **constant time**.

More Formal View

- At most $C(9, k)$ valid combinations
- Each recursive call does constant work
- Copying a valid combination costs $O(k)$

So: $O(C(9, k) \cdot k)$

Still bounded by a small constant.

Space Complexity

Recursion Stack

- Maximum depth = $k = O(k)$

Result Storage

- At most $C(9, k)$ combinations
- Each of size k : $O(C(9, k) \cdot k)$

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in **any order**.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.



Example 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Optimal:

1 How to look at the problem

Each digit maps to **multiple letters** (like a phone keypad).

So the task becomes:

“Pick **one letter per digit**, in order, and form all possible strings.”

This is a **choice-at-each-step** problem.

2 Why backtracking fits naturally

At every digit:

- You have **multiple choices** (letters mapped to that digit)
- You must try **all possibilities**

This forms a **tree of decisions**:

- Level 0 → choices for digit 1
- Level 1 → choices for digit 2
- ...
- Leaf level → one complete string

Backtracking is designed exactly for this:

choose → explore → undo

3 Meaning of index

index represents:

“Which digit am I currently processing?”

Invariant:

- Digits [0 ... index-1] are already handled
- Digits [index ... end] are yet to be handled

This keeps the recursion structured and easy to reason about.

Role of curr (StringBuilder)

curr holds:

- The **partial combination built so far**

At every step:

- One character is appended
- The problem reduces to processing the next digit

After recursion:

- The last character is removed
- This restores the state for the next choice

This explicit undo is what makes backtracking correct.

5 Base condition intuition

When the length of curr equals the number of digits:

- One valid combination is fully formed
- No more choices remain

At this point:

- The result is recorded
- The recursion stops for that branch

This guarantees that **only complete strings** are added.

Why no combinations are missed or repeated

For each digit:

- Every possible mapped letter is tried **exactly once**
- The order of digits is never changed

This ensures:

- Exhaustive exploration
- No duplicates
- Correct ordering

The recursion tree covers **all valid paths** from root to leaf.

Time Complexity

Let:

- n = number of digits
- Each digit maps to at most **4 letters**

Total combinations: $\leq 4^n$

Work per combination:

- Converting StringBuilder to String $\rightarrow O(n)$

Total time complexity: $O(n \times 4^n)$

Space Complexity

Auxiliary space:

- Recursion depth = n
- StringBuilder size $\leq n$

$O(n)$

Output space:

- Up to 4^n strings
- Each of length n

$O(n \times 4^n)$

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.

What are we trying to generate?

We want **all permutations** of an array of size n .

Meaning of permutation:

Every possible **ordering** of all elements.

So the final answers must:

- Use **every element exactly once**
 - Have **different orderings**
-

2 Core idea (mental model)

Think of permutation generation as **building the answer step by step**.

Imagine:

- You are filling positions **one by one**
- At each position, you can choose **any unused element**

So the problem becomes:

“At every step, choose one unused element and move forward.”

This is exactly why **backtracking** fits naturally.

3 Why do we need a used / flag concept?

When building a permutation:

- Once an element is placed, it **cannot be reused**
- Otherwise, duplicates or invalid permutations appear

So we maintain:

- A record of which elements are already chosen
 - This ensures **each element is used exactly once per permutation**
-

4 Recursive structure (decision tree thinking)

At any moment:

- We have a **partial permutation**
- The remaining task is to **complete it**

This leads to a natural recursive thought:

“If I fix the current position, the rest of the permutation problem is smaller.”

Each recursive call:

- Represents **one level (position)**
- Makes a choice
- Explores deeper
- Then **undoes the choice** to try others

This “choose → explore → undo” cycle is the essence of backtracking.

5 Base condition intuition

When the current permutation reaches length n :

- All positions are filled
- A valid permutation is formed

At this point:

- We **store the result**
- We do not explore further, because no positions remain

This guarantees correctness:

Only complete and valid permutations are added.

6 Why this explores all permutations?

Let's think in terms of choices:

- First position → n choices
- Second position → $n - 1$ choices
- Third position → $n - 2$ choices
- ...
- Last position → 1 choice

This exactly matches:

$$n \times (n-1) \times (n-2) \times \dots \times 1 = n!$$

So the algorithm **systematically explores the entire permutation space**, without missing or repeating any arrangement.

Time Complexity

- Total permutations of n elements = $n!$
- For **each permutation**: We spend $O(n)$ time to copy/store it

So total time:

$$O(n \times n!)$$

Space Complexity

1. Recursion depth

- Maximum depth = n
- Space = $O(n)$

2. Current permutation storage

- Holds up to n elements
- Space = $O(n)$

3. Used/flag tracking

- Boolean array of size n
- Space = $O(n)$

4. Result storage

- $n!$ permutations
- Each permutation has n elements
- Space = $O(n \times n!)$

Final Space Complexity: $O(n \times n!)$

Intuition & Thought Process (Swap-Based Permutations)

1 How to think about permutations at a high level

A permutation problem is really about **deciding what element goes at each position**.

Instead of thinking:

“Which elements should I pick next?”

This approach thinks:

“Which element should be fixed at this position?”

So the focus shifts from **choosing elements** to **fixing positions**.

2 Meaning of index

index represents the **current position** in the array that we are fixing.

- All positions **before index** are already decided
- All positions **from index onward** are still free to change

This creates a clean invariant:

At recursion level index, positions [0 ... index-1] are fixed.

3 Why swapping is the right operation

To decide which value goes at position index:

- We try **every element from index to the end**
- Swap that element into position index

Swapping is ideal because:

- It places a chosen value in the current position
- It uses **constant extra space**
- It avoids creating extra arrays or tracking structures

Each swap represents:

“Let’s assume this element belongs here.”

4 Recursive reduction of the problem

Once a value is fixed at position index, the remaining problem becomes:

“Generate all permutations of the remaining positions.”

That is the **same problem**, but smaller.

This is why recursion fits perfectly:

- Each recursive call fixes **one more position**

- The problem size reduces naturally
-

5 Base condition intuition

When `index == n`:

- Every position is fixed
- The array represents **one complete permutation**

At this moment:

- The current arrangement is recorded
- No more decisions are required

This guarantees correctness:

Only fully formed permutations are added.

6 Why backtracking (swap back) is essential

After exploring permutations with one choice at position `index`:

- The array must be restored
- So that the next choice can be tried cleanly

The pattern is always:

swap → recurse → swap back

This ensures:

- No branch affects another
- The array remains correct for future decisions

This “undo” step is the heart of backtracking.

7 Why this generates all permutations (no missing, no repeats)

At each position:

- Every remaining element is tried **exactly once**
- Each element gets a chance to occupy that position

This produces a complete **decision tree** where:

- Depth = n
- Each root-to-leaf path forms a unique permutation

With distinct elements, **no duplicates are possible.**

Time Complexity Analysis

The number of permutations of n elements is: $n!$

Work done per permutation

- Copying the array into a list takes $O(n)$
- This happens once per permutation

Total time complexity: $O(n \times n!)$

Space Complexity

Auxiliary space (excluding output)

- Recursion depth = n & Swapping uses constant extra space: $O(n)$

Output space

- $n!$ permutations & Each permutation has n elements. So, $O(n \times n!)$
-

Given a collection of numbers, nums, that might contain duplicates, return *all possible unique permutations in any order.*

Brute-Force:

By using HashSet<> to store answer in this way will only get the unique permutation & at the end will convert it into list<list<integer>>.

Time = $O(n \times n!)$

- $O(n! \times n)$ no of function will be called in total.

Space = $O(n \times n!)$

- $O(n)$ is stack space.
- $O(n \times n!)$ for storing result at the end.
- $O(n \times n!)$ for storing list<integer> inside the HashMap<List<Integer>>.

Optimal:

2 How do we think about permutations?

A permutation is built **position by position**:

- At each position, we choose **one unused element**
- We continue until all positions are filled

This naturally leads to **recursion + backtracking**.

3 Why do we need a flag / used array?

Each element comes from a specific **index**.

Even if two values are equal, they belong to **different indices**.

So we need to ensure:

Each index is used **only once** in a single permutation.

The flag[] array tracks exactly this.

4 Why sorting the array is important

Sorting groups duplicate values together.

This becomes critical because:

- Duplicate values must be **compared with their previous occurrence**
- Without sorting, we cannot reliably detect duplicates

Sorting does **not** change permutations, it only helps **control recursion**.

5 Where do duplicate permutations come from?

Duplicate permutations occur when:

- At the **same recursion level**
- We choose the **same value again**, just from a different index

Example:

[1a, 1b, 2]

Choosing 1b first at a level when 1a was skipped creates a **duplicate branch**.

6 The key duplicate-skip intuition

The condition:

`if (arr[i] == arr[i - 1] && !flag[i - 1]) → skip`

Encodes this idea:

At any position, among equal values, **only the first unused one is allowed**.

Why?

- If the previous identical element is **not used**, then choosing the current one would produce a permutation that has already been (or will be) generated.
- If the previous identical element **is already used**, then this choice represents a genuinely new arrangement.

This ensures:

- Duplicate values are used **in order**
 - No two recursion branches represent the same permutation
-

7 Why this guarantees uniqueness

- Every permutation is constructed exactly once
- No duplicate branch is allowed to start
- Yet all valid permutations are explored

So the algorithm balances:

- **Completeness** (no missing permutations)
 - **Uniqueness** (no duplicates)
-

Time Complexity

- There are **$n!$ permutations**
- Each permutation requires copying n elements
- $O(n \times n!)$

Space Complexity

- Recursion depth: $O(n)$
- `flag[]` array: $O(n)$
- Output storage: $O(n \times n!)$

Hard Problems

Given a string `s`, partition `s` such that every substring of the partition is a palindrome.
Return *all possible palindrome partitioning of s*.

Example 1:

Input: `s = "aab"`

Output: `[["a", "a", "b"], ["aa", "b"]]`

Intuition Behind the Solution

What is the problem really asking?

We are asked to **split a string into all possible partitions** such that:

- Every partition piece is a **palindrome**
- We must return **all valid ways**, not just one

This immediately tells us two things:

1. We must **try multiple possibilities**
2. We cannot decide greedily — a choice that looks good now may block valid solutions later

That's a classic sign of **backtracking**.

Core idea

At any moment, we are standing at the **start of a string s**.

From this position:

- We can cut the string at **every possible index**
- Each cut creates a **prefix**
- If the prefix is a palindrome, it is a **valid choice**
- Once chosen, we recursively solve the same problem for the **remaining suffix**

So the problem reduces to:

“Choose a palindromic prefix, then solve the rest of the string the same way.”

Why recursion is natural here

Each decision splits the problem into:

- **What I choose now** (current palindrome)
- **What remains to be solved** (rest of the string)

This self-similar structure is perfect for recursion:

- Smaller string
- Same rules
- Same goal

The recursion ends when:

- There is **no string left**
 - Meaning we have successfully formed a full valid partition
-

Role of curr and res (Conceptually)

- curr represents the **current path / partial solution**
- res represents **all completed valid solutions**

Think of curr as:

“What I have chosen so far”

When the string becomes empty:

- curr already contains **only palindromes**
- So it represents **one complete valid partition**
- We store a copy of it in res

Why backtracking is necessary

When we choose a palindrome:

- We move forward and explore deeper possibilities

But once that exploration finishes:

- We must **undo the choice**
- So that we can try a **different cut** at the same level

This is why:

- We **add** before recursion
- We **remove** after recursion

Theoretical meaning:

“Try → explore → revert → try next option”

This ensures:

- No interference between different recursive paths
- Every partition is built independently

Why the string *s* changes in recursion

At every recursive step:

- We pass a **substring of the original string**
- This substring represents the **unprocessed remainder**

So each recursive call works on:

“The part of the string that is still left to be partitioned”

This avoids the need for index tracking and keeps the logic clean.

Why palindrome checking is separated

Before making a recursive call:

- We check if the chosen prefix is a palindrome

This acts as **pruning**:

- Invalid paths are stopped early
- The recursion tree becomes smaller

Conceptually:

“Only pursue paths that can possibly lead to valid solutions”

Time Complexity

Let n be the length of the string.

Number of recursive calls

- At each position, we can make up to n cuts
- In the worst case (string like "aaaaa"), **every prefix is a palindrome**
- This leads to exploring **all possible partitions**

Number of partitions $\approx O(2^n)$

Cost of palindrome checking

- Each palindrome check takes $O(n)$ in the worst case
-

Overall Time Complexity

$O(2^n \times n)$

This comes from:

- Exponential number of partitions
 - Linear palindrome check per decision
-

Space Complexity

Recursion stack

- Maximum depth = n (each character is a partition)

Current path storage

- curr can hold up to n strings

Result storage

- Can store up to $O(2^n)$ partitions

Overall Space Complexity

$O(2^n \times n)$

(including output storage)

The n -queens puzzle is the problem of placing n queens on an $n \times n$ chessboard such that no two queens attack each other.

Given an integer n , return *all distinct solutions to the n -queens puzzle*. You may return the answer in any order.

Each solution contains a distinct board configuration of the n -queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Input: $n = 4$
Output: `[[".Q..", "...Q", "Q...", "..Q."],
["..Q.", "Q...", "...Q", ".Q.."]]`
Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above

Problem Understanding (Core Thought Process)

We must place N queens on an $N \times N$ board such that:

- No two queens share the same:

- Row
- Column
- Diagonal

Key observation:

👉 If we place **one queen per column**, we automatically avoid column conflicts.

So, the problem reduces to:

For each column, choose a row such that no conflicts occur.

This naturally leads to **Backtracking**.

1 Brute Force Approach

💡 Intuition

Step 1: Place queens column by column

- Start from column 0
- Try placing a queen in every row
- Move to next column if safe
- Backtrack if stuck

Step 2: How do we check if a position is safe?

For a queen at (row, col):

We only need to check **left side** because:

- We fill columns from left → right
- No queen exists on the right yet

So we check:

1. Same row (left side)
2. Upper-left diagonal
3. Lower-left diagonal

That's exactly what your `isValid()` does.

Theoretical Thinking

At each column:

- We branch into n possible rows.
- For each attempt, we scan:
 - $O(n)$ for row
 - $O(n)$ for diagonals

So safety check costs **$O(n)$** .

Backtracking explores many board configurations.

Complexity Analysis

◆ Number of recursive calls

In worst case:

n choices for column 0

$n-1$ for column 1

$n-2$ for column 2

...

So total possibilities $\approx$

$O(n!)$

◆ Safety check cost

Each check = $O(n)$

So total time:

$O(n! * n)$

◆ Space Complexity

- Recursion stack: $O(n)$
- Board storage: $O(n^2)$

So:

$O(n^2)$

2 Optimal Approach (Using Hashing Arrays)

Now the key improvement:

Instead of scanning the board repeatedly,
we store attack information in arrays.

Intuition Behind Optimization

When placing a queen, what conflicts can happen?

1. Same row
2. Same main diagonal
3. Same anti-diagonal

Instead of scanning the board,
we maintain 3 arrays:

`rowFlag[row]` → Is this row occupied?

`bottomDiagonal[row + col]` → Is this bottom-left to top-right diagonal occupied?

`topDiagonal[n-1 + col - row]` → Is this top-left to bottom-right diagonal occupied?

These arrays give:

$O(1)$ safety check

That's the big improvement.

Why diagonal formulas work?

Bottom diagonal:

All cells where:

$row + col = \text{constant}$

Top diagonal:

All cells where:

$col - row = \text{constant}$

To avoid negative index:

$n - 1 + (col - row)$

This maps diagonals into valid array indices.

Theoretical Thought Process

Instead of:

- Checking previously placed queens every time (costly)

We:

- Store attack state
- Update while placing
- Reset while backtracking

This converts safety check:

From $O(n) \rightarrow O(1)$

Huge improvement.

Complexity Analysis

- ◆ **Recursive tree size:** $O(n!)$

Because we still explore permutations of queen placements.

But now:

- ◆ **Safety check:** $O(1)$

So total time: $O(n!)$

◆ **Space Complexity**

- Recursion stack $\rightarrow O(n)$
- Board $\rightarrow O(n^2)$
- 3 extra arrays $\rightarrow O(n)$

So overall: $O(n^2)$

Consider a rat placed at position (0, 0) in an $n \times n$ square matrix `maze[][]`. The rat's goal is to reach the destination at position (n-1, n-1). The rat can move in four possible directions: 'U'(up), 'D'(down), 'L' (left), 'R' (right).

The matrix contains only two possible values:

- 0: A blocked cell through which the rat cannot travel.
- 1: A free cell that the rat can pass through.

Your task is to find **all possible paths** the rat can take to reach the destination, starting from (0, 0) and ending at (n-1, n-1), under the condition that the rat cannot **revisit** any cell along the same path. Furthermore, the rat can only move to adjacent cells that are within the bounds of the matrix and not blocked. If no path exists, return an **empty list**.

Note: Return the final result vector in **lexicographically smallest order**.

Input: `maze[][] = [[1, 0, 0, 0], [1, 1, 0, 1], [1, 1, 0, 0], [0, 1, 1, 1]]`

Output: `["DDRDRR", "DRDDRR"]`

Explanation: The rat can reach the destination at (3, 3) from (0, 0) by two paths - DRDDRR and DDRDRR, when printed in sorted order we get DDRDRR DRDDRR.

Core Idea

This is a classic **Backtracking + DFS** problem.

Why Backtracking?

Because:

- From each cell, we have **multiple choices (4 directions)**.
- We must explore **all possible paths**.
- If one path fails, we must **undo the move** and try another direction.

This “choose → explore → unchoose” pattern is exactly backtracking.

Thought Process (Theoretical)

1 Think in terms of a decision tree

At every cell:

- We have 4 possible moves.
- Each move leads to another decision.
- This forms a **tree of possibilities**.

We must explore all valid branches.

2 Prevent infinite loops

Since we can move in 4 directions, we might:

- Go right
- Then left
- Then right again

This creates a cycle.

So we maintain a visited[][] array:

- Mark a cell visited before exploring
 - Unmark it after returning (backtracking)
-

3 Track the path

We maintain a StringBuilder:

- Append direction before recursive call
- Remove last character after recursion

This ensures:

- Each recursive branch maintains its own path
 - Path stays correct when we backtrack
-

4 Base Case

When:

`row == N-1 && col == N-1`

We reached destination → store the current path.

Overall Algorithm Flow

For each cell:

1. Mark visited
2. Try all 4 directions:
 - Check boundary
 - Check if cell is open
 - Check if not visited
3. Append direction
4. Recurse
5. Delete last character (backtrack)
6. Unmark visited

Why This Works

Because:

- We explore every possible safe path
- We avoid revisiting cells
- We correctly restore state after each recursive call

This guarantees:

- ✓ All valid paths are found
- ✓ No duplicates
- ✓ No infinite loops

Time Complexity

Let:

N = maze dimension

In worst case:

- From each cell, we can go 4 directions.

- Maximum depth = N^2 (visit every cell once)

So worst-case time complexity:

$$O(4^{(N^2)})$$

This is exponential because:

- We're exploring all possible paths.

However:

- Due to visited array, we don't revisit cells in same path.
 - So actual practical complexity is lower than brute force.
-

Space Complexity

- 1 Recursion stack: $O(N^2)$ (max depth if path covers entire grid)
- 2 Visited array: $O(N^2)$
- 3 StringBuilder: $O(N^2)$

Final Space Complexity: $O(N^2)$

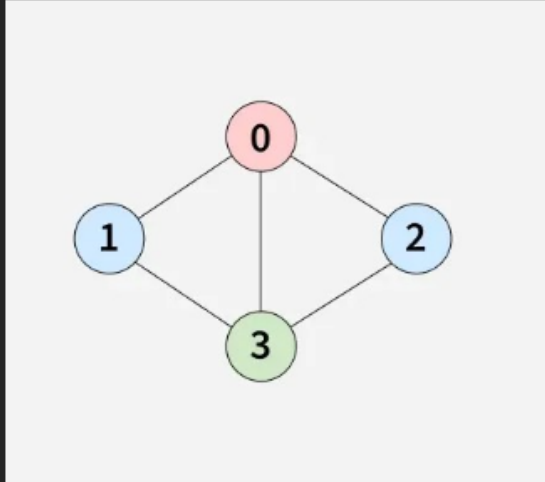
You are given an undirected graph consisting of **V** vertices and **E** edges represented by a list **edges[][]**, along with an integer **m**. Your task is to determine whether it is possible to **color the graph** using at most **m** different colors such that no two adjacent vertices share the **same color**. Return true if the graph can be colored with at most **m** colors, otherwise return false.

Note: The graph is indexed with 0-based indexing.

Input: $V = 4$, $edges[][] = [[0, 1], [1, 3], [2, 3], [3, 0], [0, 2]]$, $m = 3$

Output: true

Explanation: It is possible to color the given graph using 3 colors, for example, one of the possible ways vertices can be colored as follows:



Vertex 0: Color 1

Vertex 1: Color 2

Vertex 2: Color 2

Vertex 3: Color 3

💡 Core Idea

We solve it using **Backtracking**.

Why?

Because:

- Each vertex can take multiple possible colors.
- We must explore combinations.
- If a coloring choice leads to conflict later, we undo it and try another color.

This is exactly:

Choose → Explore → Undo (Backtrack)

🌸 Theoretical Thought Process

1 Think Sequentially

We color vertices one by one:

Vertex 0

Vertex 1

Vertex 2

...

Vertex v-1

At each vertex:

- Try color 1
 - Try color 2
 - ...
 - Try color m
-

2 Constraint Checking

Before assigning color i to vertex node, we check:

Do any of its neighbors already have color i ?

If yes $\rightarrow$ reject

If no $\rightarrow$ safe to assign

This ensures we never violate the rule:

Adjacent vertices must not have the same color.

3 Recursive Exploration

After assigning a safe color:

- Move to next vertex (node + 1)
- Continue same process

If at some vertex no color works:

- Backtrack (remove previous color)
 - Try next color at previous vertex
-

4 Stopping Condition

When:

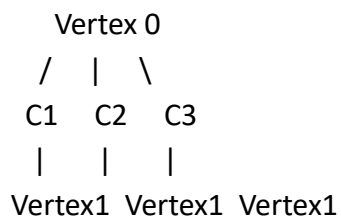
node == v

It means:

- All vertices are successfully colored.
 - Return true.
-

Decision Tree Visualization

At each vertex:



Branching factor = m

Depth = v

So we explore combinations like a tree.

Why This Works

Because:

- We explore all valid color assignments.
 - We prune(remove) invalid assignments early.
 - We stop immediately once a valid full coloring is found.
-

Time Complexity Analysis

◆ Number of States

For each vertex → m choices.

Total combinations:

$m \times m \times m \times \dots$ (v times)

So: m^v

◆ Cost Per Assignment

For each color assignment, we check all adjacent vertices.

Worst case:

- A vertex may connect to up to $v-1$ vertices.

So checking cost: $O(v)$

✅ Final Time Complexity

$$O(m^v \times v)$$

Space Complexity

1 Recursion stack depth: $O(v)$

2 Color array: $O(v)$

3 Adjacency list: $O(v + E)$

✅ Final Space Complexity: $O(v + E)$

Important Insight

Graph coloring is:

NP-Complete

Which means:

- No known polynomial-time solution for general graphs.
- Exponential time is expected.

Backtracking is the standard approach.

Why This Algorithm Is Efficient in Practice

Although worst-case is exponential:

- Early pruning avoids exploring invalid branches.
- Small m or sparse graphs (number of edges is **much smaller**) run much faster in practice.

Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

1. Each of the digits 1-9 must occur exactly once in each row.
2. Each of the digits 1-9 must occur exactly once in each column.
3. Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.

The '.' character indicates empty cells.

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Input: board =

```
[["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".",".","6","."],["8",".",".","6",".",".","3"],["4",".","8",".","3",".","1"],["7",".",".","2",".",".","6"],[".","6",".",".","2","8","."],[".","4","1","9",".","5"],[".","8",".","7","9"]]
```

Output:

```
[["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]
```

Intuition Behind the Solution

1 Understanding the Core Constraint

In Sudoku, every number placement must satisfy three rules:

- It must not already exist in the same **row**
- It must not already exist in the same **column**

- It must not already exist in the same **3×3 subgrid**

So the problem reduces to:

“For every empty cell, try placing a valid number that doesn’t violate constraints.”

2 Why Backtracking?

Sudoku is a **constraint satisfaction problem**.

At each empty cell:

- We don’t know the correct number.
- But we can try all possible numbers (1–9).
- If a number leads to a contradiction later, we undo it.

This naturally suggests:

Try → Recurse → If it fails → Undo (Backtrack)

Backtracking works because:

- We explore possibilities depth-first.
 - The moment a partial solution becomes invalid, we stop exploring that branch.
 - This avoids exploring all 9^81 possibilities blindly.
-

3 High-Level Thought Process

Step 1: Find an Empty Cell

Scan the board.

- If no empty cell remains → Sudoku is solved.
- If found → attempt to fill it.

This ensures we always work on the next unresolved position.

Step 2: Try All Possible Numbers (1–9)

For the chosen empty cell:

- Try placing '1' to '9'
- For each number, check if it is valid using `isSafe()`.

This represents:

"Can I legally place this number here?"

Step 3: Validate Placement (isSafe Logic)

To validate:

- Check row
- Check column
- Check 3×3 subgrid

The 3×3 subgrid trick:

```
int rowStart = (row / 3) * 3;
```

```
int colStart = (col / 3) * 3;
```

This finds the top-left corner of the box.

Then the mapping:

```
rowStart + (i / 3)
```

```
colStart + (i % 3)
```

Allows us to iterate through all 9 cells of the box in a single loop.

This is just a mathematical way to traverse a 3×3 block.

Step 4: Recursive Exploration

If placement is valid:

- Place number
- Recursively attempt to solve the rest of the board

If recursion returns false:

- That means future cells couldn't be filled
- So undo this choice (backtrack)

This reflects:

"This decision was wrong. Let's try another number."

Step 5: Termination Condition

If the board has no empty cells:

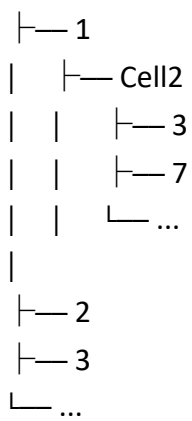
- We have a complete valid configuration
- Return true and unwind recursion

Conceptual Recursion Tree

Each empty cell creates up to 9 branches.

Example structure:

Cell1



But due to constraint pruning, many branches are cut early.

This pruning is what makes the solution feasible.

Time Complexity Analysis

Let:

- $N = 9$ (board size)
- E = number of empty cells

Worst Case:

For each empty cell:

- Try up to 9 numbers
- For each number, `isSafe()` takes $O(9)$

So worst-case time:

$$O(9^E)$$

In the absolute worst case:

$$O(9^{81})$$

But in reality:

- Constraints prune heavily
 - Practical performance is much faster
-

isSafe Complexity

isSafe() does:

- 9 row checks
- 9 column checks
- 9 subgrid checks (compressed into one loop)

So:

$$O(9) = O(1)$$

Constant time.

Overall Practical Complexity

With pruning:

- Typically behaves closer to exponential with strong reduction
 - Most real Sudoku puzzles solve very fast
-

Space Complexity

Recursion Stack

In worst case:

- Depth = number of empty cells
- Maximum depth = 81

So:

$$O(81) = O(1)$$

(Constant since board size fixed)

2 Board Storage

Board is 9×9:

$$O(81) = O(1)$$

The set [1, 2, 3, ..., n] contains a total of $n!$ unique permutations.

By listing and labeling all of the permutations in order, we get the following sequence for $n = 3$:

1. "123"
2. "132"
3. "213"
4. "231"
5. "312"
6. "321"

Given n and k , return the k^{th} permutation sequence.

```
Input: n = 3, k = 3
Output: "213"
```

Brute Force:

Core Idea Behind This Approach

The goal is:

Generate permutations in lexicographical order and stop when we reach the k -th one.

Instead of using math (factorial system), this approach:

- Generates permutations one by one
- Counts them
- Returns the k-th permutation when found

It simulates the natural permutation generation process.

Thought Process Step-by-Step

1 Build the Permutation Gradually

We maintain:

- `StringBuilder sb` → current permutation being built
- `boolean[] used` → which numbers are already taken

At each step:

- Try all numbers from $1 \rightarrow n$
- If a number is unused:
 - Choose it
 - Mark it used
 - Recurse

This builds permutations depth-first.

2 Base Case

`if(sb.length() == n)`

When length becomes n:

- We formed one complete permutation
- Increase count
- If `count == k`, return this permutation
- Otherwise, return empty string (continue searching)

So we are effectively saying:

“Every time we reach depth n, we have one permutation.”

3 Why count[] Instead of int?

Java passes primitives by value.

So we use:

```
int[] count = {0};
```

So that recursive calls share and update the same counter.

4 Backtracking Mechanism

After recursion:

```
sb.deleteCharAt(sb.length()-1);  
used[i] = false;
```

This step is crucial.

It means:

Undo the last decision and try the next number.

This explores the entire permutation tree.

5 How the Recursion Tree Looks

For n = 3:

```
      ""  
     / | \  
    1  2 3  
   / \ / \ / \  
  12 13 21 23 31 32  
  |  |  |  |  |  |  
 123 132 213 231 312 321
```

This generates permutations in lexicographic order because:

- We always try smaller numbers first.
 - DFS traversal ensures correct ordering.
-

6 Why It Works

Because:

1. DFS naturally explores permutations lexicographically (since we try $1 \rightarrow n$).
 2. Every complete path corresponds to one permutation.
 3. We stop early when count reaches k .
-

Time Complexity Analysis

Let's analyze carefully.

Total Possible Permutations: $n!$

Because:

- First position $\rightarrow n$ choices
 - Second $\rightarrow n-1$
 - ...
 - Last $\rightarrow 1$
-

Work Per Permutation

To build one permutation:

- Append operations $\rightarrow O(1)$
- Delete operations $\rightarrow O(1)$
- Final toString $\rightarrow O(n)$

So each permutation costs roughly: $O(n)$

Worst Case

If $k = n!$ (last permutation):

We generate all permutations.

So: $O(n \times n!)$

Space Complexity

1 Recursion Stack

Depth = $n = O(n)$

2 `used[]` array: $O(n)$

3 `StringBuilder`: $O(n)$

Final Space Complexity: $O(n)$

Optimal

Core Thought Process

◆ Key Observation

If we have numbers:

1, 2, 3, 4

Total permutations: $4! = 24$

Now observe something very important:

If we fix the first digit:

- Starting with **1** $\rightarrow (3!) = 6$ permutations
- Starting with **2** $\rightarrow (3!) = 6$ permutations
- Starting with **3** $\rightarrow (3!) = 6$ permutations
- Starting with **4** $\rightarrow (3!) = 6$ permutations

So permutations are divided into **blocks of size $(n-1)!$** .

This is the key idea.

Main Idea

Instead of generating permutations one by one:

We determine which “block” the k -th permutation lies in.

Each block has size:

$$(n - 1)!$$

So:

$$\text{First digit index} = \frac{k}{(n - 1)!}$$

This immediately tells us:

- Which number should be placed first.

Then we:

- Remove that number
- Repeat the process for remaining digits

This is based on the **Factorial Number System**.

Why We Do $k = k - 1$

Permutation indexing is naturally 0-based in math logic.

But problem gives k as 1-based.

So we convert:

$$k = k - 1$$

So calculations become easier.

Step-by-Step Conceptual Example

Suppose:

$$n = 4$$

$$k = 17$$

$$\text{Total permutations} = 24$$

$$\text{Block size} = 3! = 6$$

Step 1

$k = 16$ (after $k-1$)
 $\text{index} = 16 / 6 = 2$

So pick 3rd element from sorted list.

Remaining numbers shrink.

Step 2

Now remaining digits = 3
New block size = $2! = 2$

$k = 16 \% 6 = 4$
 $\text{index} = 4 / 2 = 2$

Pick next digit.

Repeat until list becomes empty.

🧩 What Your Code Is Doing

Initialization

- `arr` → stores available digits
 - `fact` → stores $(n-1)!$
 - Convert k to 0-based
 - Use `StringBuilder` for result
-

Loop Logic

Each iteration:

1. Determine index:

$$\text{index} = k / \text{fact}$$

2. Append that number.
3. Remove it from available numbers.
4. Reduce k :

$$k = k \% fact$$

5. Reduce factorial:

$$fact = fact / remainingSize$$

Repeat until no numbers remain.

Why This Works

Because permutations are grouped in equal-sized blocks:

$$(n - 1)!, (n - 1)!, (n - 1)!, \dots$$

Each first digit fixes a block.

Then inside each block, second digit fixes smaller blocks of size:

$$(n - 2)!$$

And so on.

We are basically navigating a **decision tree directly using division instead of DFS traversal**.

Time Complexity Analysis

Let's analyze carefully.

Step 1 Initial Setup

- Building array $\rightarrow O(n)$
 - Calculating factorial $\rightarrow O(n)$
-

Step 2 Main Loop

Loop runs exactly n times.

Inside loop:

- `arr.get(index)` $\rightarrow O(1)$
- `arr.remove(index)` $\rightarrow O(n)$ (because shifting in ArrayList)

So each iteration costs $O(n)$

Total:

$$O(n \times n) = O(n^2)$$

✓ Final Time Complexity

$$O(n^2)$$

📦 Space Complexity

- `ArrayList` $\rightarrow O(n)$
- `StringBuilder` $\rightarrow O(n)$
- No recursion

So:

$$O(n)$$

Given an $m \times n$ grid of characters board and a string word, return true if word exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

A	B	C	E
S	F	C	S
A	D	E	E

Input: board = `[["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]]`, word = `"ABCCED"`

Output: `true`

Intuition Behind the Solution

1 What Type of Problem Is This?

The question says:

- Letters must be adjacent (up/down/left/right)
- A cell cannot be reused
- We must check if the word exists

These are strong signals of:

 Backtracking + DFS on Grid

Because:

- We try building the word step-by-step
- If a path fails → we undo and try another direction

2 Core Thought Process

Imagine you are standing on a cell.

You ask:

“Can I build the remaining part of the word starting from here?”

So at every cell:

1. If character matches the current word index
 2. Try exploring 4 directions
 3. If any direction completes the word → success
 4. Otherwise → backtrack
-

Why Start From Every Cell?

Because the word can begin anywhere in the grid.

So we try:

For every cell (i, j):

Try building word starting from here

Why Do We Mark Visited?

We cannot reuse the same cell in a single path.

So when we use a cell:

- Mark it visited
- Explore neighbors
- Unmark it when returning (backtracking)

This ensures:

- No cycles
 - No repeated usage
 - Other paths can still use it later
-

How the Recursion Thinks

At any call:

`helper(row, col, index)`

Means:

“Can I match `word[index...end]` starting from `board[row][col]`?”

Step 1: If character doesn't match → stop

Because this path is invalid.

Step 2: If this is the last character → success

That means we matched the full word.

Step 3: Mark visited

Because this cell is now part of the current path.

Step 4: Try 4 directions

Up

Right

Down

Left

If any works → return true.

Step 5: Backtrack

If none works:

- Unmark visited
- Return false

This allows other paths to reuse the cell.

 What Is Actually Happening? (Recursion Tree View)

From one starting cell:

At index 0 → up to 4 choices

At index 1 → up to 3 choices (cannot go back)

At index 2 → up to 3 choices

...

So it forms a branching tree.

Time Complexity Analysis

Let:

- m = number of rows
- n = number of columns
- L = length of word

Step 1: We try starting from every cell

$$m \times n$$

Step 2: From each cell we explore paths

At each character:

- Maximum 4 directions at first step
- After that, at most 3 directions (can't go back)

So:

$$O(4 \times 3^{L-1})$$

Which simplifies to:

$$O(3^L)$$

Final Time Complexity

$$O(m \times n \times 3^L)$$

Worst-case theoretical upper bound is sometimes written as:

$$O(m \times n \times 4^L)$$

But practically it behaves closer to:

$$O(m \times n \times 3^L)$$

Space Complexity

Two parts:

1 Visited matrix

$$O(m \times n)$$

2 Recursion stack

Maximum depth = L

$$O(L)$$

Total Space

$$O(m \times n + L)$$

If we modify the board instead of using vis, space becomes:

$$O(L)$$